Solution Requirements

Date	02 July 2026
Team ID	CCAP-01
Project Name	Credit Card Approval Prediction
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (what the system should do).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (how the system should be).

The tables below fill out the descriptions and details for each category for the Credit Card Approval Prediction system.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	Users sign up and log in through the Flask app's signup/login pages, with session-based authentication and (optionally) Google login for identity verification.	High
2	Authorization levels	Two roles: Applicant/User can submit applications and view their own prediction history; Bank Analyst/Admin can review all predictions, batch results, and model information.	Medium
3	External interfaces	Consumes application_record.csv and credit_record.csv as source data; loads the trained scikit-learn/XGBoost model and preprocessor via Joblib; supports optional Google OAuth login.	Medium
4	Transactions processing	Handles single-applicant prediction requests and batch CSV uploads, running the preprocessing.py and predict.py pipeline and returning an Approved/Rejected result.	High
5	Reporting	Generates a prediction history dashboard and lets users download each prediction as a PDF (via ReportLab) or Excel (via OpenPyXL) report.	High
6	Business rules, etc.	Target label is derived from credit_record.csv: statuses 2, 3, 4, or 5 become Rejected; clean/minor statuses C, X, 0, or 1 become Approved, as implemented in preprocessing.py.	High
7	Compliance to laws or regulations	Loads secrets (SECRET_KEY) from a local .env file rather than hardcoding them; the project explicitly notes that a production system would need fairness review, monitoring, and data-privacy compliance checks.	Medium
8	Other	Includes a model info page, a feature reference page, a custom 404 error page, and input validation with error handling on all forms.	Low

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Single-applicant predictions use the pre-trained, saved model so results return in near real time.	Prediction returned in under 2 seconds per applicant.
2	Scalability	The app is deployed via Gunicorn and Flask (see Procfile) and can scale horizontally by adding more workers/dynos.	Batch CSV uploads of up to 1,000 applicant rows processed without timeout.
3	Security & Data Privacy	Sensitive configuration (SECRET_KEY) is loaded from a local .env file and never committed; login/signup uses hashed credentials.	No plaintext secrets in source code; .env excluded from version control.
4	Reliability & Availability	An automated pytest suite covers input validation, credit-rule behavior, downloads, history management, and login/signup, run via CI (.github/workflows/ci.yml).	All automated tests pass before each deployment; environment auto-repairable via Review-Project.ps1.
5	Usability & Accessibility	A simple, styled web UI (templates/ and static/style.css) provides clear forms, validation messages, and a custom 404 page.	Core actions (predict, download report, view history) completed within 3 clicks.
6	Other	Modular codebase (preprocessing.py, train.py, predict.py, app.py) with pinned requirements.txt, Dockerfile, and runtime.txt supports maintainability and portability.	Runs consistently on Windows, macOS, and Linux with the pinned dependency versions.